

Large Language Models and Generative AI

AI Bootcamp in Biotechnology – Rutgers and KAIMRC Academy

W. Evan Johnson, Ph.D.
Professor, Division of Infectious Disease
Director, Center for Data Science
Associate Director, Center for Biomedical Informatics and Health AI
Rutgers University – New Jersey Medical School

2026-09-15

How it works

Recurrent Neural Networks

- ▶ A type of neural network for **sequential data**
- ▶ Maintains **memory of previous inputs** using hidden states
- ▶ Commonly used for:
 - ▶ Natural Language Processing (NLP)
 - ▶ Time Series Forecasting
 - ▶ Speech Recognition

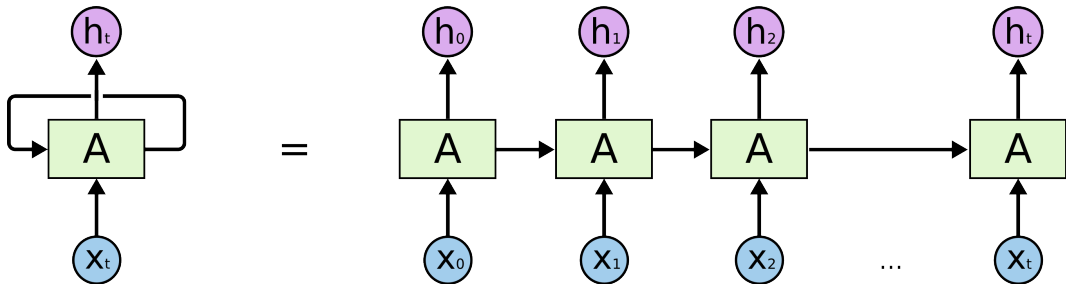
Why Not Feedforward Networks?

- ▶ Standard neural networks assume **independent inputs**
- ▶ But sequences (like text) have **temporal dependencies**
- ▶ Example:
 - ▶ Sentence: *"The cat sat on the ____."*
 - ▶ You need previous words to predict the blank

RNN Architecture

- ▶ Loops over time: same weights applied at each time step
- ▶ Input at time t : x_t
- ▶ Hidden state: $h_t = f(Wx_t + Uh_{t-1} + b)$
- ▶ Output: $y_t = g(Vh_t + c)$

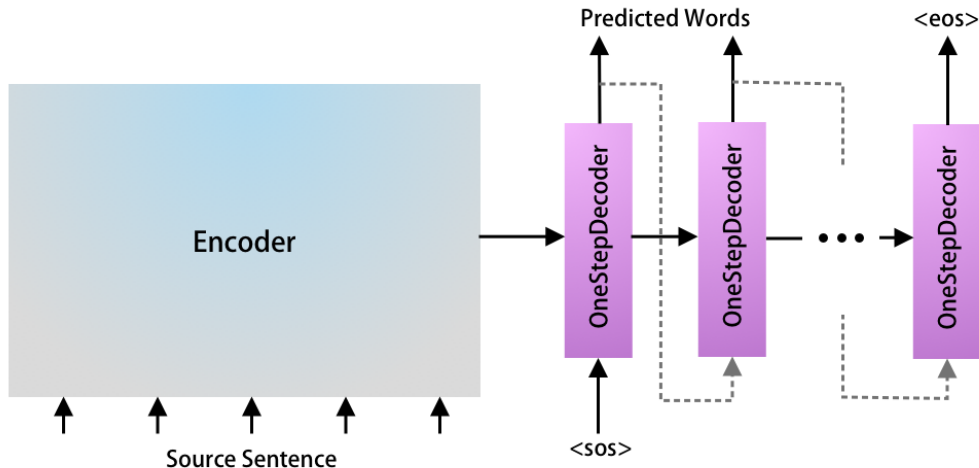
Diagram of RNN (Unrolled)



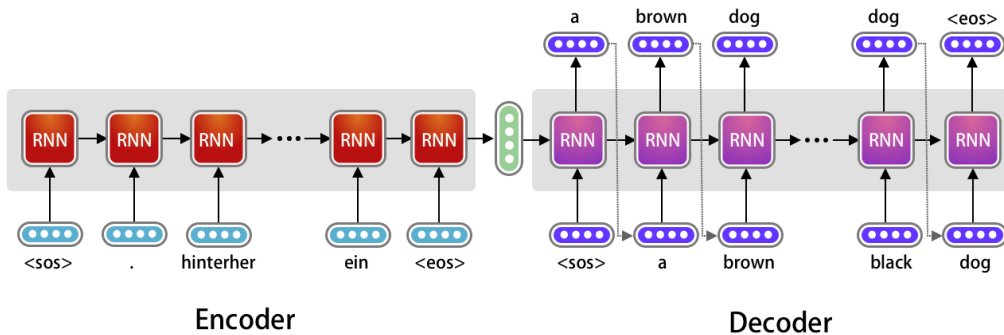
Challenges of RNNs

- ▶ Vanishing/exploding gradients during training
- ▶ Hard to learn long-term dependencies
- ▶ Solutions:
 - ▶ LSTM (Long Short-Term Memory)
 - ▶ GRU (Gated Recurrent Unit)

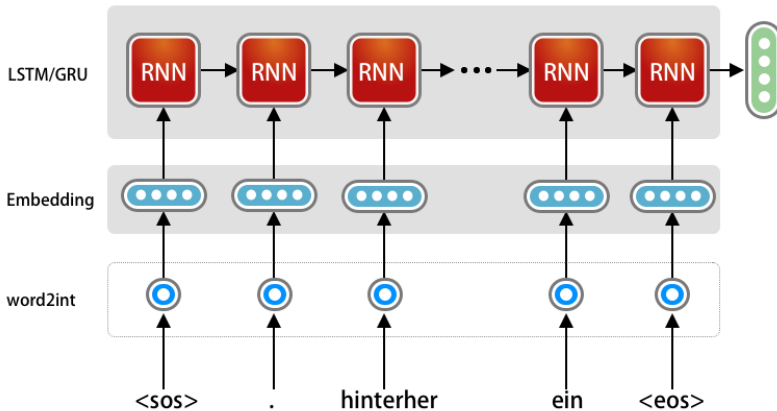
RNN for Language Translation



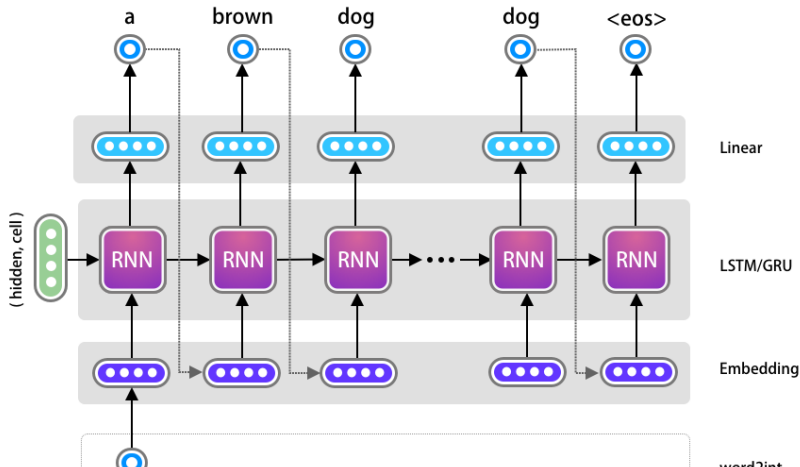
RNN for Language Translation



RNN for Language Translation



RNN for Language Translation



From RNNs to Transformers

- ▶ RNNs process sequences **one step at a time** – slow, and long-range context decays
- ▶ Transformers process the **entire sequence at once**
- ▶ Introduced in *“Attention is All You Need”* (Vaswani et al., 2017)

Key Idea: Attention

- ▶ Attention lets the model **focus on the relevant parts** of the input
- ▶ Computes attention scores between all pairs of tokens
- ▶ Every position can look at every other position, directly

Self-Attention (Scaled Dot Product)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Queries, Keys, and Values are all learned projections of the same sequence. The softmax turns “how relevant is each token to this one” into weights.

Multi-Head Attention

- ▶ Multiple attention “heads” allow the model to learn different relationships
- ▶ Outputs from each head are concatenated and linearly transformed

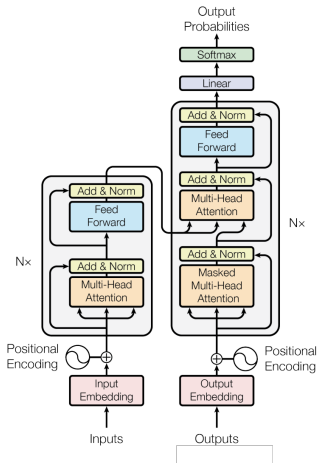
Positional Encoding

- ▶ Transformers have no recurrence or convolution
- ▶ Positional encoding injects **order information** into the input embeddings
- ▶ Common method: use sine/cosine functions of different frequencies

Transformer Architecture

BERT

Encoder



GPT

Decoder

Why this mattered

- ▶ **Parallel:** the whole sequence at once, so training scales with hardware
- ▶ **Long-range:** no decay over distance
- ▶ **General:** the same architecture handles text, images, and biological sequences

Scale then did the rest. The architecture did not need to change much.

What an LLM actually does

It predicts the next token.

Everything else – summarizing, translating, answering, writing code – is that one operation, conditioned on enough context and trained at enough scale that the behavior becomes useful.

This is worth remembering when you are deciding whether to trust an output.

From prediction to agency

A shift in what we ask AI to do

Predictive AI – *Forecast and classify.* Maps inputs to outputs.
“Based on the past, what is likely to happen next?”

Generative AI – *Create.* Learns a distribution and samples from it. “Can you produce something new that looks like what you’ve seen?”

Agentic AI – *Act.* Plans, uses tools, and works toward a goal.
“Here is the objective – figure out how to get there.”

Powering Generative AI

Large Language Models: built on the transformer, using self-attention to predict the next token. Strong at context, summarization, and translation.

Diffusion Models: generate images by gradually removing noise from a random signal. The engine behind text-to-image tools.

Comparative Analysis

Feature	Predictive AI	Generative AI	Agentic AI
Primary goal	Forecast accuracy & classification	Content creation	Task completion
Interaction	Input → class/value	Prompt → response	Goal → workflow
Output	Numbers, labels, probabilities	Text, images, code	Completed tasks, API calls, file edits
Reasoning	Statistical correlations	Pattern matching & probability	Planning, reflection, tool selection

The Architecture of Agency

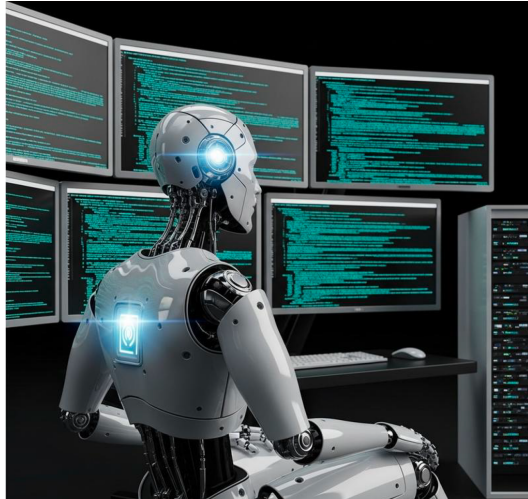
Perception: the agent gathers information – reads a file, queries a database, receives a prompt.

Reasoning: usually an LLM. Breaks the goal into a plan, decides which tool to use, reflects on what came back.

Action: the execution layer – APIs, code interpreters, queries.

The loop repeats until the goal is met or the agent asks for help.

Agentic AI



Biomedical applications

Where these tools actually earn their place

- ▶ **Unstructured text at scale** – notes, reports, the literature
- ▶ **Code you would otherwise write by hand**
- ▶ **Structuring the unstructured** – extraction into analyzable tables
- ▶ **Multi-step workflows** with a human at the decision points

Note what is *not* on this list: deciding what is true.

Literature and evidence synthesis

- ▶ Screening abstracts against inclusion criteria
- ▶ Extracting study characteristics into a structured table
- ▶ Drafting a first-pass summary of a body of work

The gain is triage speed. The requirement is that every extracted value stays traceable to its source – which means the output is a table with citations, not a paragraph.

Extraction from clinical text

- ▶ Pulling stated findings out of radiology and pathology reports
- ▶ Normalizing free-text fields to a shared data dictionary
- ▶ Flagging records that need human review

This is where a federated setting helps: the text never leaves the site, and only the structured result is shared.

Code generation for analysis

An LLM can turn an analysis request into R or Python that runs against a known data dictionary.

This is the use with the clearest payoff and the clearest danger. The code is plausible whether or not it is correct, so it needs the same validation you would demand of a collaborator's script – read it, test it, check the result against something you already know.

An agentic framework for federated analysis

From our RePORT International work:

- ▶ Investigator states the research question
- ▶ The agent proposes cohort, eligibility, exposure, outcome
- ▶ It generates metadata-aware code
- ▶ Code executes **inside the site's environment** – no data moves
- ▶ Results return for review

Human review at key decision points

The agent automates routine execution.

The investigator retains every scientific choice.

This is the design principle that makes the rest defensible. Automating the typing is not the same as automating the judgment.

Limits

What these models are bad at

- ▶ **Confident error.** Fluency is not accuracy, and the two look identical on the page.
- ▶ **Provenance.** A generated citation may not exist.
- ▶ **Arithmetic and counting**, unless a tool does it.
- ▶ **Knowing what it does not know.**

The reproducibility problem returns

An LLM will happily write you an analysis with the feature selection outside the cross-validation loop.

It has read a great deal of published code, and a great deal of published code leaks. Everything from the evaluation session applies here – with less visibility, because you did not write it.

Generated code is a draft, not a result.

A working rule

Use it where verification is cheap.
Be careful where it is not.

Questions & Discussion

Thank you for your attention.

Hands-on practice

Everything in this session has a matching workshop you can work through at your own pace:

`workshop_capstone`

- ▶ An end-to-end analysis of RePORT India TB data
- ▶ Elastic net, SVM, and random forest compared on the same problem

All workshops are in the course repository, with the R Markdown source and a rendered HTML version.

Session Info

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.6.2
##
## Matrix products: default
## BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/libBLAS.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  LAPACK version 3.12.1
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Asia/Qatar
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] knitr_1.51
##
## loaded via a namespace (and not attached):
## [1] compiler_4.5.2   fastmap_1.2.0    cli_3.6.6        tools_4.5.2
## [5] htmltools_0.5.9  otl_0.2.0        rstudioapi_0.19.0 yml_2.3.12
## [9] rmarkdown_2.31   xfun_0.58        digest_0.6.39    rlang_1.2.0
## [13] evaluate_0.22.1
```